

GLITZ - Cahier des Charges :

1. Hiérarchie des Rôles :

-  **Super ADMIN:** Contrôle total
-  **Owner (user par défaut) :** Gère ses auto-écoles

2. TABLEAU des Fonctionnalités par Rôle :

Fonctionnalité	Super ADMIN	Owner (Par défaut)
Visibilité des données	TOTAL	Ses Auto-écoles
Gérer les tarifs du SaaS	✓	✗
Gérer les plans	✓	✗
Modifier IBAN / Mandat SEPA	✓	✓
Créer une auto-école	✓	✓
Modifier Profil & Mot de passe	✓	✓
Consulter, Télécharger factures	✓	✓

2. Chemin de Mise en Service (Onboarding SaaS)

ÉTAPE 1 : INSCRIPTION ↴

- Création du compte et Configuration de la 1ère auto-école
-  Choix du plan (Avec / Sans Engagement / Need-to-contact)
-  Signature du mandat SEPA (Active l'accès au SaaS)

ÉTAPE 2 : CRÉATION DES AGENCES ↴

-  Possibilité d'ajouter une nouvelle Auto-école avec le choix du plan
-  Saisie des informations de la nouvelle agence
-  Choix du plan pour cette nouvelle agence
-  Mise à jour du prélèvement mensuel global

3. Architecture Technique (NestJS & MongoDB)

A. Stratégie Multi-Tenancy

Comme un utilisateur peut posséder plusieurs auto-écoles, nous utiliserons une approche de **base de données partagée avec isolation logique** :

- Chaque document (Élève, Planning, Facture) possède une propriété **schoolId**.
- Un **Intercepteur NestJS** ou un **Middleware** injecte automatiquement le **schoolId** dans les requêtes pour garantir qu'un Admin d'agence ne voit pas les données d'une autre agence.

on met en place un intercepteur automatique, tourne avant chaque requete :

- détecte quelle école concernée
- inject schoolid dans le requête
- Les demandes à BD → obligé filtre par schoolid

B. Schéma de Données (Mongoose)

1. COLLECTION : USERS

Champ	Type	Contrainte	Description
_id	ObjectId	Auto-généré	
nom	String	Requis	
prenom	String	Requis	
tel	String	Requis	
email	String	Unique / Requis	

mot_de_passe	String	Requis	Mot de passe sécurisé (Hashé).
role	String	Enum	SUPER_ADMIN ou OWNER .
ownedSchools	Array[ID]	Référence	Liste des IDs des écoles appartenant à ce gérant.

2. COLLECTION : SCHOOLS

Champ	Type	Contraintes	Description
_id	ObjectId	Unique	
ownerId	ObjectId	Ref Users	Lien vers le gérant.
name	String	Requis	Nom de l'auto-école.
num_siret	String	Requis	SIRET (14 chiffres).
adresse	String	Requis	
ville	String	Requis	
code_postal	String	Requis	

plan_type	String	Enum	ENGAGEMENT/ /SANS_ENGAGEMENT/ NEED_TO_CONTACT
payment_status	String	Enum	

3. 🇫🇷 COLLECTION : PAYMENTS

Champ	Type	Contrainte	Description
schoolId	ObjectId		
ownerId	ObjectId		
montant	Number	Requis	Somme payée en euros.
Montant	Number		
status	String	Enum	(paid/pending/overdue)
date_paiement	Date		Date et heure de la transaction.

a rajouter : date expiration abonnement,

4. Sécurité et Implémentation NestJS

A. Guards & Decorators

Empêcher qu'un utilisateur (même administrateur) puisse accéder aux données d'une auto-école à laquelle il n'est pas rattaché.

garantit que , l'utilisateur ne peut pas voir les données des autres auto écoles

Utilisation de décorateurs personnalisés pour protéger les routes API :

TypeScript

@Roles(Role.ADMIN)

@UseGuards(RolesGuard)

@Get('students')

findAllStudents(@Query('schoolId') schoolId: string) {

 // Le guard vérifie si l'utilisateur a le rôle ADMIN pour CE schoolId

}

Principe

- On pose une étiquette (@Roles) sur chaque route pour dire quels rôles sont autorisés (ex. @Roles('ADMIN')).
- Un filtre (RolesGuard) s'exécute automatiquement avant chaque route.
- Ce filtre vérifie deux choses :
 1. L'utilisateur est bien connecté.
 2. Il a le bon rôle pour l'école concernée par la requête.
- Le numéro de l'école (schoolId) est récupéré soit dans l'URL, soit dans l'en-tête, soit depuis le profil de l'utilisateur.
- Si l'utilisateur n'est pas autorisé pour cette école → accès refusé (erreur 403).

B. Validation des données

Garantir l'intégrité des données entrantes et assurer une traçabilité complète des actions critiques (création, modification, suppression) sur les entités métier (abonnements, auto-écoles, etc.).

- **Délégation de validation** : Utilisation de **class-validator** pour vérifier que les IBAN saisis sont valides avant l'envoi vers le prestataire SEPA.
- **Logs d'audit** : Traçabilité de qui a modifié quel abonnement ou supprimé quelle auto-école.

Brique	Technologie	Rôle
Validation	class-validator + DTO	Filtrer les données invalides dès l'entrée (ex. IBAN).

Traçabilité document	Champs *By, *At	Connaître rapidement le dernier responsable d'un enregistrement.
Audit trail	Collection AuditLog	Historiser toutes les actions critiques pour analyse et conformité.

5. Workflow de Création (User Journey)

1. **Inscription** : L'utilisateur crée son compte (devient **Owner** par défaut).
2. **Onboarding** : * Il crée sa première auto-école.
 - Il choisit son abonnement.
 - Il signe le mandat SEPA.
3. **Expansion** :
 - Il peut cliquer sur "Ajouter une agence".
 - Le système lui demande de choisir un nouveau plan (ou le même).
 - L'abonnement est rattaché à cette nouvelle entité.
4. **Gestion** :
 - Il invite des collaborateurs en leur assignant des rôles (Admin, Moniteur) par agence

🎯 OBJECTIF VISÉ : LE "LOGIN" OPÉRATIONNEL

À la fin de cette étape, tu dois pouvoir utiliser **Postman** pour :

1. **Créer un compte** : Envoyer les infos et vérifier que l'utilisateur est bien créé.
2. **Sécuriser** : Vérifier que le mot de passe est illisible (hashé) dans MongoDB.
3. **S'authentifier** : Recevoir un jeton **JWT** qui prouve l'identité et le rôle (Owner ou Admin).

🔧 RÉPARTITION DES TÂCHES ET FICHIERS À REMPLIR

👤 DEV 1 : L'Architecte (Infrastructure & Connexion)

Sa mission : Faire en sorte que le serveur "parle" à la base de données et valide les données.

- **src/main.ts** :
 - Activer le **ValidationPipe** global pour que les DTO fonctionnent.
 - Ajouter le préfixe **/api** à toutes les routes.
- **src/app.module.ts** :
 - Importer **MongooseModule.forRoot()** avec ton lien MongoDB.
 - Importer **AuthModule** et **UsersModule**.
- **.env** :
 - Définir la variable **MONGO_URI** et **JWT_SECRET**.

```
MONGODB_URI="mongodb+srv://Vercel-Admin-Glitz-master:G5XCgFZ7U4dFqE0P@glitz-master.mdpccbr.mongodb.net/?retryWrites=true&w=majority/"
```

🛡️ DEV 2 : Le Gardien (Data & Sécurité)

Sa mission : Créer l'objet "Utilisateur" et s'assurer que personne ne voit son mot de passe.

- **src/users/schemas/user.schema.ts** :
 - Définir le schéma avec : nom, prenom, email (unique), mot_de_passe, role (enum).
- **src/users/users.service.ts** :
 - Coder la méthode **create()** : elle doit utiliser **Bcrypt** pour hasher le mot de passe avant de sauvegarder.
 - Coder la méthode **findOneByEmail()** pour la future connexion.
- **src/users/users.module.ts** :
 - Déclarer le modèle Mongoose et exporter le **UsersService**.

🔑 DEV 3 : Le Guichetier (Logique & API)

Sa mission : Coder les routes de connexion et de création de compte.

- `src/auth/dto/register.dto.ts & login.dto.ts` :
- `src/auth/auth.service.ts` :
 - Coder `validateUser()` : comparer le mot de passe saisi avec celui en BDD.
 - Coder `login()` : générer le token **JWT** avec le `role` à l'intérieur.
- `src/auth/auth.controller.ts` :
 - Créer la route `@Post('register')` (appelle le service Users).
 - Créer la route `@Post('login')` (appelle le service Auth).

Développeur	Focus	Résultat attendu
DEV 1	Config	Le serveur tourne sur le port 3000 sans erreur de BDD.
DEV 2	Sécurité	Dans MongoDB, les mots de passe ressemblent à <code>\$2b\$10....</code>
DEV 3	Flux	Un <code>POST</code> sur <code>/api/auth/login</code> renvoie un JSON avec un <code>access_token</code> .

POSTMAN TEST

1. Inscription (Signup)

URL : POST <http://localhost:3000/api/auth/signup>

JSON

```
{  
  "nom": "Ziad",  
  "prenom": "F",  
  "tel": "0601015304",  
  "email": "ton-email@gmail.com",  
  "mot_de_passe": "MonMotDePasseSecurise123",  
  "nom_auto_ecole": "Auto-Ecole Glitz",  
  "num_siret": "12375678901234",  
  "adresse": "10 rue de la Paix",  
  "ville": "Paris",  
  "code_postal": "75000",  
  "plan_type": "SANS_ENGAGEMENT"  
}
```

2. Connexion (Login) — *Nouveau*

Cette étape est indispensable pour récupérer l'[access_token](#) et le [refresh_token](#). **URL**
: POST <http://localhost:3000/api/auth/login>

JSON

```
{  
  "email": "ton-email@gmail.com",  
  "mot_de_passe": "MonMotDePasseSecurise123"
```

}

- **Attendu :** 201 Created ou 200 OK.
- **Action :** Copie la valeur de "refresh_token" reçue dans la réponse.

3. Rafraîchir le Token (Refresh Token) — *Nouveau*

Ce test vérifie que ton `RefreshTokenModel` et ton schéma fonctionnent correctement en base de données. **URL** : POST `http://localhost:3000/api/auth/refresh`

JSON

```
{
  "refresh_token": "COLLER_ICI_LE_TOKEN_DU_LOGIN"
}
```

- {
- "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2OTk1ZGNlYmEwOTFjNjkwMTk5MzRjYzEiLCJpYXQiOiE3NzE0MjkyMTYsImV4cCI6MTc3MTQzMjgxNn0.9azIoInN3xRvbacH4nJ-hA82iib5dYsX8NMPTzxik3U",
- "refresh_token": "e2820825-7ca8-4a91-82ff-8dce07786494"
- } **Attendu : 200 OK** avec une nouvelle paire de tokens.

4. Mot de passe oublié (Forgot Password)

URL : POST http://localhost:3000/api/auth/forgot-password

JSON

```
{ "email": "ton-email@gmail.com" }
```

- **Vérif Mailtrap** : Récupère le code à 6 chiffres reçu par mail.

5. Réinitialisation (Reset Password)

URL : POST <http://localhost:3000/api/auth/reset-password>

JSON

```
{  
  "token": "LE_CODE_MAILTRAP",  
  "nouveau_mot_de_passe": "12345678NewPassword"  
}
```

6. Login Final (Validation) — *Nouveau*

Vérifie que tu peux te connecter avec le **nouveau** mot de passe. **URL :** POST <http://localhost:3000/api/auth/login>

JSON

```
{  
  "email": "ton-email@gmail.com",  
  "mot_de_passe": "12345678NewPassword"  
}
```

Test 1 - Envoyer un message de contact

POST <http://localhost:3000/api/contact>

```
{  
  "nom": "Jean Dupont",  
  "email": "jean@example.com",  
  "telephone": "0612345678",  
  "entreprise": "Mon Entreprise",  
  "message": "J'aimerais avoir plus d'informations sur vos services"  
}
```

Test 2 - Récupérer tous les messages

GET <http://localhost:3000/api/contact>

Test 3 - Sans les champs optionnels

POST <http://localhost:3000/api/contact>

```
{  
  "nom": "Marie",  
  "email": "marie@test.com",  
  "message": "Juste un message simple"  
}
```

Test 4 - Récupérer tous les messages

GET <http://localhost:3000/api/contact>